

Static Keyword(Module-26)

Assignment

Q.1 The **static** keyword in Java is used to define members (fields or methods) of a class that belong to the class itself rather than to any specific instance of the class. These members are shared among all instances of the class and can be accessed without creating an instance of the class.

Why Do We Need the **static** Keyword?

Memory Efficiency: Static members are stored in a common memory location, saving memory since they are not duplicated for each instance.

Global Access: Static members can be accessed directly using the class name, making them convenient for utility or constant values.

Shared Data: Static fields can be used to share data across all instances of a class.

Utility Methods: Static methods are useful for performing tasks that do not depend on instance-specific data.

Example:

```
class Counter {
    // Static variable to count the number of objects created
    static int count = 0;

    // Constructor
    Counter() {
        count++; // Increment the count whenever an object is created
    }

    // Static method to display the count
    static void displayCount() {
        System.out.println("Number of objects created: " + count);
    }
}

public class Main {
    public static void main(String[] args) {
        // No objects created yet
        Counter.displayCount(); // Accessing static method without creating an instance

        // Create instances of Counter
        Counter c1 = new Counter();
        Counter c2 = new Counter();
        Counter c3 = new Counter();
    }
}
```

```
// Static variable is shared, so the count is updated across all instances
Counter.displayCount(); // Output: Number of objects created: 3
    }
}
```

Q.2 Class Loading in Java

Class loading is the process by which the Java Virtual Machine (JVM) dynamically loads classes into memory when required during the execution of a program. This is managed by the **ClassLoader**, which is responsible for locating, loading, and linking class files.

Phases of Class Loading

1. **Loading:**
 - The `.class` file (bytecode) is located and loaded into the JVM's memory.
 - A `Class` object representing the class is created in the method area of **memory**.
2. **Linking:** This step prepares the class for execution.
 - Verification: Ensures the bytecode complies with the JVM's specifications (e.g., no illegal operations or memory violations).
 - Preparation: Allocates memory for static fields and assigns default values (e.g., `0` for integers, `null` for objects).
 - Resolution: Converts symbolic references in the bytecode (e.g., `methodA`) into direct references to memory locations.
3. **Initialization:**
 - Executes the class's static initializers (`static {}` blocks) and static variable assignments.
 - After this phase, the class is ready for use.

How Does a Java Program Execute?

The execution of a Java program involves several stages:

1. Compilation

- The Java source code (`.java` files) is compiled by the Java Compiler (`javac`) into bytecode (`.class` files).
- Bytecode is platform-independent and can run on any JVM.

2. Execution

The JVM executes the program in the following steps:

1. JVM Initialization:

- The `java` command starts the JVM.
- The JVM locates the `main` class (the class containing the `public static void main(String[] args)` method).

2. Class Loading:

- The Bootstrap ClassLoader loads core Java classes (e.g., `java.lang.String`).
- The Application ClassLoader loads user-defined classes.
- Dependencies are loaded on demand.

3. Bytecode Interpretation:

- The JVM reads bytecode line by line and interprets it into machine instructions.
- The Just-In-Time (JIT) Compiler optimizes frequently used bytecode by converting it into native machine code for faster execution.

4. Runtime Execution:

- The JVM executes the program, starting with the `main` method.
- During execution:
 - Objects are created in heap memory.
 - Local variables and method calls are stored in stack memory.

5. Garbage Collection:

- Unused objects are automatically removed by the JVM's Garbage Collector, freeing memory.

Q.3 Local variables cannot be declared as `static`.

- The `static` keyword in Java is used for class-level variables and methods, not for local variables inside a method.

Example:

This code will result in an error:

```
public class Test {  
  
    public void someMethod() {  
  
        static int count = 0; // Compilation Error  
  
        count++;  
  
    }  
  
}
```

Q.4 In Java, the static block (or static initializer block) is executed before the `main()` method because it is part of the class loading and initialization process.

Class Loading Process:

1. Class Loading:

- When a Java program is run, the JVM starts by loading the class specified in the `java` command.
- Example: `java Example` triggers the JVM to load the `Example` class.

2. Class Initialization:

- During class initialization, static variables and static blocks are executed in the order they appear in the class definition.
- Static blocks are executed only once when the class is loaded, before any instance of the class is created or any static method (like `main()`) is called.

3. Method Execution:

- After initializing static members and executing static blocks, the JVM looks for the `main()` method to start the program's execution.

3. Purpose of Static Blocks

- **Initialization Logic:** Static blocks are often used to initialize static variables or perform one-time setup before the program starts.
- **Resource Management:** They can load configuration files, initialize static data structures, or set up logging frameworks.

Q.5 static method in Java is also referred to as a class method because it belongs to the class rather than any specific instance of the class. This behavior is fundamentally different from instance methods, which require an object of the class to be called.

Here's why static methods are called class methods:

1. Belongs to the Class, Not Instances

Static methods are associated with the class itself and not with any object of the class.

- You can call a static method using the class name, without creating an object of the class.

Example:

```
class Example {
```

```
static void staticMethod() {  
    System.out.println("This is a static (class) method.");  
}  
}
```

```
public class Test {  
    public static void main(String[] args) {  
        Example.staticMethod(); // Called using the class name  
    }  
}
```

2. Shared Across All Instances

A static method operates at the class level and doesn't depend on instance-specific data.

Since it doesn't require an object, it can be shared across all instances of the class.

Example:

```
class Example {  
    static void showClassName() {  
        System.out.println("This method is shared by all instances of the class.");  
    }  
}
```

```
public class Test {  
    public static void main(String[] args) {  
        Example.showClassName(); // No object creation needed  
    }  
}
```

3. Cannot Access Instance Variables Directly

Static methods cannot directly access instance variables or instance methods because they do not belong to any specific instance.

They can only work with:

- Static variables
- Static methods
- Parameters passed to them

Example

```
class Example {  
  
    int instanceVar = 10;  
  
    static int staticVar = 20;  
  
    static void staticMethod() {  
  
        // System.out.println(instanceVar); // ERROR: Cannot access instanceVar  
  
        System.out.println(staticVar); // OK: Accessing staticVar  
  
    }  
  
}
```

Q.6 In Java, static blocks (also known as static initialization blocks) are used to execute code when the class is loaded into memory, before any instance of the class is created or any static method is called. They are primarily used for class-level initialization and are executed only once per class.

Uses of Static Blocks in Java

- Static blocks are commonly used to initialize static variables when the initialization requires complex logic.
- This is particularly useful when the initialization cannot be achieved through a simple assignment.
- Static blocks are used to load resources, configuration files, or perform setup tasks at the time of class loading.
- Static blocks allow you to execute certain actions before the `main()` method runs. This can be useful for logging or pre-execution tasks.
- A static block ensures that certain actions are performed when the class is loaded, even if no object is created or no static method is called.

Q.7 The key difference between static variables and instance variables lies in their scope, lifetime, memory allocation, and how they are used in Java. Here's a detailed comparison:

Comparison Table: Static vs. Instance Variable

Feature	Static Variable	Instance Variable
Belongs To	Class (shared across all instances)	Object (unique to each instance)
Memory Allocation	Allocated in the method area once per class load	Allocated in the heap memory for each object
Access	Accessed using <code>ClassName.variableName</code> or <code>objectName.variableName</code>	Accessed only through <code>objectName.variableName</code>
Shared Across Objects	Yes, shared by all instances of the class	No, each instance has its own copy
Initialization	Initialized once when the class is loaded	Initialized when the object is created
Keyword	Declared using the <code>static</code> keyword	No keyword required
Lifetime	Exists as long as the class is loaded in memory	Exists as long as the object is alive
Use Case	Used for class-level constants or shared resources	Used for storing object-specific data

Direct Access	Can be accessed directly in static methods	Cannot be accessed directly in static methods
----------------------	--	---

Q.8 The key difference between static members and non-static members in Java lies in their association with the class or an instance, scope, access, and memory management. Here's a detailed comparison:

The key difference between **static members** and **non-static members** in Java lies in their **association with the class or an instance, scope, access, and memory management**. Here's a detailed comparison:

Comparison Table: Static vs. Non-Static Members

Feature	Static Members	Non-Static Members
Belongs To	The class (shared across all instances)	Individual objects (unique to each instance)
Access Method	Accessed using <code>ClassName.memberName</code> or <code>object.memberName</code>	Accessed only through <code>object.memberName</code>
Requires an Object?	No, can be accessed without creating an object	Yes, requires an object to access
Memory Allocation	Allocated once in the method area when the class is loaded	Allocated in the heap memory for each object

Direct Access in Static Context	Yes, static members can be accessed directly in static methods	No, non-static members cannot be accessed directly in static methods
Shared or Unique?	Shared among all objects	Unique to each object
Lifetime	Exists as long as the class is loaded	Exists as long as the object exists
Initialization	Initialized when the class is loaded	Initialized when an object is created
Use Case	Class-level constants, shared resources, utility methods	Object-specific properties and behaviors

